



SPACEGHOSTKILLA

CLOUD SECURITY QUICK AUDIT

Offensive mindset. Defensive engineering.

Version 1.2 · September 2026 · spaceghostkilla.com

Before you start

This is a baseline audit of 25 controls that often decide whether a cloud incident is a non-event or a breach. For one small AWS account or Azure subscription, expect roughly an hour once audit access is set up. It won't make you compliant with anything. It shows you where to look first.

Ground rules

- Only audit environments you own or are explicitly authorized to assess. Run it against **one account, subscription or zone at a time**.
- The workflow is **non-destructive**: it changes no configuration and no resources. Two steps do create state, and both are marked **[STATE]**:
 - AWS credential report generation (setup step A) creates or refreshes the account's single IAM credential report.
 - The *optional* alert test in check 11 creates labelled GuardDuty sample findings. Only run it if you're authorized to.
- Record every check as **PASS**, **FAIL**, **N/A** (the control doesn't apply, with a reason), or **UNKNOWN** (you couldn't collect the evidence: missing permission, license, skipped region). An UNKNOWN is **not** a PASS.
- **Empty output is not automatically a PASS**. A check passes only when its commands completed without errors. Multi-command blocks finish by printing `DONE: ...` or `UNKNOWN: ...`. Any error message, a missing `DONE` line, or an `UNKNOWN:` line makes that check UNKNOWN.
- **Evidence is sensitive**. Credential reports, role lists and exposure data are a map for an attacker. Keep evidence files readable only by you (the setup uses `umask 077`), never paste secret values into notes, and delete the evidence when the audit ticket closes, or keep it in access-controlled storage per your retention policy.
- **Run the companion scripts** (`sgk-quick-audit-v1.2.zip`, delivered with this PDF) rather than copying commands out of the PDF. PDF viewers can drop indentation, and the embedded Python depends on it. Each script is named after its check, e.g. `check-02-aws.sh`. The code in this PDF is identical, for reading and review.
- Commands assume a Linux-style shell with the AWS CLI v2, Azure CLI, `jq`, `curl` and `dig`. **AWS CloudShell** and **Azure Cloud Shell** have all of these. Run AWS commands in every region you use (add `--region <region>`), since most EC2 and GuardDuty results are per region.

Access you need

Platform	Role / permission	Covers	Without it
AWS	SecurityAudit managed policy (includes <code>iam:GenerateCredentialReport</code>)	Checks 01–18	Mark affected checks UNKNOWN
AWS	<code>lambda:GetFunctionConfiguration</code> , <code>ecs:DescribeTaskDefinition</code>	Check 06 (workload config)	Check 06 is UNKNOWN for Lambda/ECS
AWS	<code>guardduty:CreateSampleFindings</code> (optional)	Check 11 alert test	Test the alert path another way
Azure	Reader + Security Reader on the subscription	Checks 07–16	UNKNOWN
Entra ID	Global Reader directory role	Checks 01–02 (role members, Conditional Access)	UNKNOWN
Entra ID	Microsoft Graph <code>AuditLog.Read.All</code> + <code>User.Read.All</code> , and an Entra ID P1/P2 license	Check 05 (<code>signInActivity</code>)	Check 05 sign-in evidence is UNKNOWN
Cloudflare	Read-only API token or dashboard access for the zone	Checks 19–22	UNKNOWN

Severity and response

Severity is a default. Raise it when the resource is internet-facing or holds sensitive data, and lower it only with a written reason.

Severity	Response to a FAIL (or a CRIT UNKNOWN)
CRIT	Contain now. Start the same day: enable MFA, remove the key, close the port, then investigate whether it was already used.
HIGH	Fix within 30 days, with an owner and a date.
MED	Fix within 90 days.

Setup (AWS)

A. Get the IAM credential report [STATE]

Checks 02, 04 and 05 read this report (companion script: `setup-a-aws.sh`; run it in the directory where you'll run those checks). The block below waits at most 60 seconds for the report, checks that the file is really a credential report, and stops with an error instead of leaving you with an empty file that looks like a PASS. The parentheses run it in a subshell, so an error won't close your terminal.

BASH

```
(  
  set -euo pipefail
```

```

umask 077
state=""
for attempt in $(seq 1 20); do
    state=$(aws iam generate-credential-report --query State --output text)
    [ "$state" = "COMPLETE" ] && break
    sleep 3
done
[ "$state" = "COMPLETE" ] || { echo "ERROR: report not ready after 60s (state: $state)" >&2; exit 1; }
aws iam get-credential-report --query Content --output text | base64 -d > credential-report.csv
python3 - credential-report.csv <<'EOF'

import csv, sys
need = {"user", "arn", "user_creation_time", "password_enabled", "password_last_used", "mfa_active",
        "access_key_1_active", "access_key_1_last_rotated", "access_key_1_last_used_date",
        "access_key_2_active", "access_key_2_last_rotated", "access_key_2_last_used_date"}

try:
    with open(sys.argv[1], newline="") as f:
        reader = csv.DictReader(f, strict=True)
        missing = need - set(reader.fieldnames or [])
        if missing:
            sys.exit("ERROR: not a credential report (missing " + ", ".join(sorted(missing)) + ")")
        rows = 0
        for row in reader:
            if None in row or None in row.values():
                sys.exit(f"ERROR: malformed report row at line {reader.line_num}")
            rows += 1
except csv.Error as e:
    sys.exit(f"ERROR: malformed credential report: {e}")
print(f"OK: {rows} principals in credential-report.csv")
EOF
)

```

Only continue to checks 02, 04 and 05 if this printed OK . An error (for example `AccessDenied` , or a malformed report) means those checks are UNKNOWN. The report is parsed with a real CSV parser (Python's `csv` module) and columns are read **by header name**. IAM user names may legally contain commas, which break naive comma splitting, and AWS can add columns.

1. Identity & Access

01 **CRIT** Root / Global Admin accounts have MFA and no standing use

The root user (AWS) or Global Administrator (Entra ID) can bypass almost every other control. Password-only access to these accounts turns one phished credential into total compromise.

- **AWS:** `aws iam get-account-summary --query 'SummaryMap.[AccountMFAEnabled,AccountAccessKeysPresent]'`. The expected result is `[1, 0]`: root MFA on, no root access keys.
- **Azure (Global Reader):** Entra admin center → Roles & admins → Global Administrator. List the members and confirm each is covered by a phishing-resistant MFA requirement (Conditional Access authentication strength).
- **PASS:** root has MFA and **no access keys**; Global Admins are few (typically 2–4 named people plus break-glass accounts) and all require strong MFA.
- **If FAIL:** contain now. Remove root keys and enable MFA today, then review CloudTrail or sign-in logs for recent use.

02 **CRIT** MFA enforced for every human user

- **AWS (IAM users, needs setup A):**

```
python3 - credential-report.csv <<'EOF'
import csv, sys
with open(sys.argv[1], newline="") as f:
    for row in csv.DictReader(f, strict=True):
        if row["password_enabled"] == "true" and row["mfa_active"] == "false":
            print(row["user"])
EOF
```

BASH

This prints console users **without** MFA. Empty output is a PASS only if setup A printed `OK`.

- **Azure (Global Reader):** Confirm a Conditional Access policy requires MFA for *All users* (break-glass accounts excluded and monitored), or that Security Defaults are on for tenants without Conditional Access.
- **PASS:** no console user lacks MFA, and new human access can only be created with MFA enforced (ideally through SSO, check 03).

03 **HIGH** Humans use SSO / federation, not long-lived IAM users

Each IAM user with a password and keys is a separate credential lifecycle that you have to manage.

- **AWS:** `aws iam list-users --query 'length(Users)'`. Then check whether IAM Identity Center is in use: `aws sso-admin list-instances`.
- **PASS:** humans sign in through Identity Center or your IdP. IAM users exist only for documented legacy or break-glass needs.

04 **HIGH** No access keys older than 90 days or unused for 90+ days

- **AWS (needs setup A):** list every *active* key with its rotation and last-use dates:

```
python3 - credential-report.csv <<'EOF'
import csv, sys
with open(sys.argv[1], newline="") as f:
    for row in csv.DictReader(f, strict=True):
        for k in ("1", "2"):
            if row[f"access_key_{k}_active"] == "true":
                print(row["user"], f"key{k}", "rotated=" + row[f"access_key_{k}_last_rotated"],
                    "last_used=" + row[f"access_key_{k}_last_used_date"], sep="\t")
EOF
```

Output is tab-separated. `last_used=N/A` means the key has **never** been used.

- **PASS:** no active key was rotated more than 90 days ago, and none is unused (or never used) after 90 days. Better still, workloads use roles, not keys (check 06).

05 HIGH Stale identities are disabled

Former employees and forgotten test users are the quietest way in.

- **AWS (needs setup A):**

```
python3 - credential-report.csv <<'EOF'
import csv, sys
with open(sys.argv[1], newline="") as f:
    for row in csv.DictReader(f, strict=True):
        if row["password_enabled"] == "true":
            print(row["user"], "created=" + row["user_creation_time"],
                "password_last_used=" + row["password_last_used"], sep="\t")
EOF
```

Read `password_last_used` carefully. A date is the last sign-in. `no_information` means the password has never been used, or AWS has no record of it. (`N/A` would mean the user has no password, so such users don't appear in this list.) Treat a user as stale only after checking **both** console use (above) **and** access-key use (check 04).

- **Azure (Graph `AuditLog.Read.All` + `User.Read.All` , Entra ID P1/P2):**

```
az rest --method get \
  --url "https://graph.microsoft.com/v1.0/users?
  \?$select=userPrincipalName,accountEnabled,signInActivity"
```

A 403 or missing `signInActivity` means sign-in evidence is UNKNOWN, not "never signed in". Compare enabled accounts against HR records either way.

- **PASS:** every enabled identity has been used in the last 90 days **or** maps to a documented current person or workload.

06 **HIGH** Workloads use roles / managed identities, not embedded keys

These checks read **configuration metadata and variable names only**. Never print, copy or save the *values* of environment variables. They may be live secrets.

- **AWS EC2** without an instance profile (instances like this often carry keys in files or user-data): `aws ec2 describe-instances --query 'Reservations[].Instances[?!IamInstanceProfile][].InstanceId'`
- **AWS Lambda**: list environment variable **names** that look like credentials:

```
(  
  set -o pipefail  
  fns=$(aws lambda list-functions --query 'Functions[].FunctionName' --output text) || exit 1  
  for f in $fns; do  
    names=$(aws lambda get-function-configuration --function-name "$f" \  
      --query 'keys(Environment.Variables | `{}')' --output text) \  
      || { echo "UNKNOWN: $f (could not read configuration)"; continue; }  
    printf '%s\n' $names | grep -iE 'key|secret|token|passw|cred' | sed "s|^|$f: |" || true  
  done  
  true  
) && echo "DONE: Lambda functions checked" || echo "UNKNOWN: could not list Lambda functions"
```

Each flagged line is `function: VARIABLE_NAME`. The block ends with `DONE` or `UNKNOWN`. The API returns variable values to your session even though `--query` prints only names. Run this in a trusted shell, without `--debug`, and don't save the raw API response.

- **AWS ECS**: variable names in a task definition (`secrets[]` entries are references to Secrets Manager or Parameter Store, which is the good pattern): `aws ecs describe-task-definition --task-definition <family:revision> --query 'taskDefinition.containerDefinitions[].environment[].name'`
- **Manual follow-up (tightly scoped)**: for each flagged name, the workload owner confirms in the console whether the value is a long-lived cloud credential. Record only "yes/no" in the audit, never the value.
- **Azure**: App Services and Functions should use a managed identity: `az webapp identity show -g <rg> -n <app>`.
- **PASS**: no workload holds long-lived cloud keys in its configuration, instance files or images. Credentials come from roles, managed identities or a secret store.

07 **HIGH** No wildcard admin for principals that don't need it

- **AWS**: find who holds `AdministratorAccess`: `aws iam list-entities-for-policy --policy-arn arn:aws:iam::aws:policy/AdministratorAccess` Also review customer-managed policies that grant `"Action": "*" ON "Resource": "*" .`
- **Azure**: `az role assignment list --all --query "[?roleDefinitionName=='Owner' || roleDefinitionName=='Contributor']" -o table`
- **PASS**: every admin assignment has a named owner and a reason. CI/CD and apps are scoped to what they deploy.

2. Logging & Detection

08 **CRIT** Audit logging is on in every region, and log storage is tamper-resistant

Without control-plane logs you can't answer "what happened?" after an incident.

- **AWS:** `aws cloudtrail describe-trails --query 'trailList[].[Name,IsMultiRegionTrail,LogFileValidationEnabled]'`, then `aws cloudtrail get-trail-status --name <trail> ( IsLogging: true )`.
- **Azure:** the Activity Log is on by default but only kept for 90 days. Check for a diagnostic setting that exports it: `az monitor diagnostic-settings subscription list`.
- **PASS:** a multi-region trail is logging with log-file validation on, delivered to storage that app and admin roles can't delete from (ideally a separate log-archive account).

09 **HIGH** Logs are retained for at least 1 year

- **AWS:** check the lifecycle rules on the CloudTrail bucket: `aws s3api get-bucket-lifecycle-configuration --bucket <bucket>`.
- **Azure:** Log Analytics workspace retention: `az monitor log-analytics workspace show -g <rg> -n <ws> --query retentionInDays`.
- **PASS:** at least 365 days, or whatever your contracts or regulators require if longer.

10 **HIGH** Threat detection is enabled

- **AWS:** `aws guardduty list-detectors` returns a detector ID in each region you use (or it's enabled organization-wide).
- **Azure:** Defender for Cloud plans: `az security pricing list -o table` shows `Standard` on the resource types you run.
- **PASS:** it's enabled **and someone receives the findings** (check 11).

11 **HIGH** High-severity findings and root/admin sign-ins alert a human

An alert that nobody reads is the same as having no alert.

- **Read-only check:** confirm there is a route from GuardDuty/Defender/CloudTrail to email, chat or a pager. For example, an EventBridge rule on root `ConsoleLogin` or GuardDuty findings (`aws events list-rules`), or Defender email notifications (`az security contact list`).
- **Optional alert test [STATE], only if authorized:** `aws guardduty create-sample-findings --detector-id <id>`. This creates clearly labelled sample findings, changes no resources, and needs `guardduty:CreateSampleFindings`, which audit roles normally lack.
- **PASS:** a route exists **and** a test alert reached a person within 15 minutes. If you couldn't test delivery, record UNKNOWN.

3. Data & Storage

12 **CRIT** Account-level public access block is on for object storage

- **AWS:** `aws s3control get-public-access-block --account-id <id>`. All four settings should be `true`.
- **Azure:** storage accounts with public blob access allowed: `az storage account list --query "[?allowBlobPublicAccess].name"`.
- **PASS:** the block is on account-wide, and any public bucket is an intentional, documented exception (for example, fronted by a CDN with origin access control).

13 **HIGH** Encryption at rest is on, and you know who controls the keys

- **AWS:** S3 encrypts new objects by default. Check EBS default encryption per region: `aws ec2 get-ebs-encryption-by-default`. For RDS: `aws rds describe-db-instances --query 'DBInstances[].[DBInstanceIdentifier,StorageEncrypted]'`.
- **Azure:** storage and managed disks are encrypted by default. Check whether sensitive stores need customer-managed keys.
- **PASS:** there are no unencrypted databases or volumes, and for regulated data the key policy is reviewed.

14 **HIGH** Snapshots and backups are not shared publicly

- **AWS (EBS):** `aws ec2 describe-snapshots --owner-ids self --restorable-by-user-ids all --query 'Snapshots[].SnapshotId'`
- **AWS (RDS):** check each manual snapshot's `restore` attribute:

```
(  
  set -o pipefail  
  snaps=$(aws rds describe-db-snapshots --snapshot-type manual \  
    --query 'DBSnapshots[].DBSnapshotIdentifier' --output text) || exit 1  
  for s in $snaps; do  
    values=$(aws rds describe-db-snapshot-attributes --db-snapshot-identifier "$s" \  
      --query "DBSnapshotAttributesResult.DBSnapshotAttributes[?  
AttributeNames=='restore'].AttributeValues[]" \  
      --output text) || { echo "UNKNOWN: $s (could not read attributes)"; continue; }  
    printf '%s\n' $values | grep -qx all && echo "PUBLIC: $s"  
  done  
  true  
) && echo "DONE: RDS snapshots checked" || echo "UNKNOWN: could not list RDS snapshots"
```

BASH

Repeat with `describe-db-cluster-snapshots / describe-db-cluster-snapshot-attributes` for Aurora.

- **PASS:** in every region you use, the EBS command returns an empty list without an error, **and** the RDS block prints `DONE` with no `PUBLIC:` or `UNKNOWN:` lines.

15 **HIGH** Backups exist, are isolated, and a restore has been tested

Ransomware and bad deploys are recovery problems. If the attacker's credentials can delete your backups, you don't really have them.

- **AWS:** `aws backup list-backup-plans`. Check for Vault Lock or a cross-account copy.
- **Azure:** `az backup vault list`, then check that soft delete and immutability are enabled.
- **PASS:** critical data is backed up, copies sit outside the blast radius of production admin credentials, and a restore was **actually tested** in the last 6 months. No restore evidence means UNKNOWN, not PASS.

4. Network Exposure

16 **CRIT** No admin or database ports open to the internet

Group-level filters aren't enough: a group with one public HTTPS rule may also have private-only rules. List each **individual rule** whose source is the whole internet (IPv4 `0.0.0.0/0` or IPv6 `:::/0`):

- **AWS:**

```
(  
  set -o pipefail  
  aws ec2 describe-security-groups --output json | jq -r '  
    .SecurityGroups[] | .GroupId as $g | .IpPermissions[] |  
    ([.IpRanges[]?.CidrIp | select(. == "0.0.0.0/0")]  
      + [.Ipv6Ranges[]?.CidrIpv6 | select(. == ":::/0")]) as $src |  
    select($src | length > 0) |  
    [$g, (if .IpProtocol == "-1" then "all" else .IpProtocol end),  
      (.FromPort // "all"), (.ToPort // "all"), ($src | join(" ")) | @tsv'  
) && echo "DONE: security groups checked" || echo "UNKNOWN: could not read security groups"
```

BASH

Each line is: group, protocol, port range, public source. A protocol of `all` means every port. The block must end with `DONE`. `UNKNOWN` means the listing failed, and no rows then does **not** mean nothing is exposed. Then check whether a flagged group is actually reachable. List the network interfaces using it, and whether they have a public IP: `aws ec2 describe-network-interfaces --filters Name=group-id,Values=<sg-id> --query 'NetworkInterfaces[]. [NetworkInterfaceId,Association.PublicIp,Description]' --output text`

- **Azure:**

```
(
  set -o pipefail
  az network nsg list -o json | jq -r '
    .[] | .name as $nsg | .securityRules[] |
    select(.access == "Allow" and .direction == "Inbound") |
    ([.sourceAddressPrefix] + (.sourceAddressPrefixes // []))
      | map(select(. == "*" or . == "Internet" or . == "0.0.0.0/0")) as $src |
    select($src | length > 0) |
    [$nsg, .name, .protocol,
      (if (.destinationPortRange // "") != "" then .destinationPortRange
        else (.destinationPortRanges // [] | join(",")) end),
      ($src | join(" "))] | @tsv'
) && echo "DONE: NSGs checked" || echo "UNKNOWN: could not read NSGs"
```

- **PASS:** every internet-open rule is on your **approved exposure list**, which records the service, port, owner and reason. Admin and database ports (22, 3389, 3306, 5432, 1433, 6379, 9200, 27017) and all-ports rules are never on it. Admin access goes through SSM Session Manager, Bastion or a VPN.
- A public rule on a group with no public network interface is a latent risk: record it as **MED** and remove it anyway.

17 **HIGH** Instance metadata is protected (IMDSv2)

SSRF plus IMDSv1 leads to stolen instance credentials. This is one of the most common cloud breach chains.

- **AWS:**

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[?MetadataOptions.HttpTokens!='required'][].InstanceId"
```

- **PASS:** the list is empty in every region, and new instances default to `HttpTokens=required` (account-level default: `aws ec2 get-instance-metadata-defaults`).

18 **MED** You have an up-to-date inventory of what's internet-facing

- List public IPs, load balancers, API gateways and DNS records, then compare them against your approved exposure list (check 16). For example:

```
aws ec2 describe-addresses --query 'Addresses[][PublicIp,InstanceId,NetworkInterfaceId]'
aws elbv2 describe-load-balancers --query "LoadBalancers[?Scheme=='internet-facing'].DNSName"
```

- **PASS:** every public endpoint has a known owner and purpose. Anything unexplained gets investigated this week.

5. Edge, DNS & Web

19 **HIGH** DNS has no dangling records (subdomain takeover)

A CNAME that points at a deleted bucket, app or SaaS tenant lets someone else claim the target and serve content on your domain.

- Export your zone. For each CNAME, check that the target still resolves **and** still belongs to you: `dig +short CNAME <sub>.example.com`. Look for targets that return `NXDOMAIN`, and for provider "not found" pages (S3 `NoSuchBucket`, Azure `*.azurewebsites.net` 404s, GitHub Pages 404s).
- **PASS:** every CNAME points at a resource you control.

20 **HIGH** Email domain is protected against spoofing

Decide which case applies first. The right answer depends on who actually sends mail as the domain.

- **Look up the current records:** `dig +short TXT example.com` (SPF), `dig +short TXT _dmarc.example.com` (DMARC).
- **Domain that sends mail:** list every service that sends as the domain (mail provider, marketing, billing, ticketing). Use the **exact SPF and DKIM records each provider generates**. There must be only **one** SPF record, so merge the includes into it. Publish DMARC at `p=none` with a report address (`rua=`), confirm in the reports that every legitimate sender passes aligned SPF or DKIM, then move to `p=quarantine` and later `p=reject`.
- **Receive-only or forwarding domain** (for example, a domain that only uses an email-forwarding service): use the provider-generated SPF/DKIM records. Enforce DMARC once reports confirm nothing legitimate sends as the domain.
- **Domain that never sends or receives mail:** `v=spf1 -all` and DMARC `p=reject`.
- **PASS:** DMARC is at enforcement (`quarantine` or `reject`) with alignment verified in reports. A documented rollout still at `p=none` is a FAIL with an owner and a date, not a PASS.

21 **HIGH** Web apps you operate aren't directly exposed at the origin

Applies to sites and APIs served by an origin *you* run (VMs, containers, load balancers), especially ones with logins, APIs or payments.

- **Cloudflare:** the web records are **Proxied** (orange cloud), managed WAF rules are on, and SSL/TLS mode is **Full (strict)**, never *Flexible*. DNS-only (grey-cloud) records bypass Cloudflare's proxy entirely, so WAF and SSL settings don't apply to them.
- Confirm the origin only accepts traffic from the CDN: a tunnel, Authenticated Origin Pulls, or a firewall allowlist of CDN ranges.
- **N/A with a documented exception:** static sites on managed hosting that you don't operate (e.g. GitHub Pages, Netlify, S3 + CloudFront with origin access control). The platform is the origin. Record the platform and why a WAF isn't needed.
- **PASS:** requesting the origin IP directly does not serve the app, or the site qualifies for the documented exception.

22 **MED** HTTPS is enforced and security headers are set

- `curl -sI http://example.com` should redirect to `https://`.

- `curl -sI https://example.com` should show `Strict-Transport-Security`, a `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, and framing protection (`frame-ancestors` in CSP, or `X-Frame-Options`).
- **PASS:** HTTP redirects to HTTPS, and apps that handle sessions send HSTS and CSP. Hosts that can't set headers (some static platforms) are a documented exception for HSTS; add CSP via a `<meta>` tag where possible.

6. Secrets, Pipelines & Dependencies

23 **CRIT** No secrets in source code or CI logs

- Scan every repo, **including history**, with output redacted so secrets aren't printed: `gitleaks detect --source . --log-opts="--all" --redact` (or `trufflehog git file://. --no-verification`).
- Enable your platform's push protection (GitHub secret scanning + push protection).
- **PASS:** there are no live secrets in history. Anything ever committed has been **rotated**, not just deleted.
- **If FAIL:** contain now. Rotate or revoke the credential first, then clean up the history.

24 **HIGH** CI/CD deploys with short-lived, scoped credentials

Pipelines usually hold the most powerful credentials an organization has.

- Check whether pipelines authenticate with **OIDC federation** (GitHub Actions → AWS `sts:AssumeRoleWithWebIdentity`, or Azure workload identity federation) instead of stored keys.
- Confirm the trust policy is restricted to your repo **and branch or environment** (`token.actions.githubusercontent.com:sub`). A trust policy of `repo:org/*`, or one with no `sub` condition, is a finding.
- Confirm production deploys require a protected branch or environment approval.
- **PASS:** no static cloud keys are stored in CI secrets, and the deploy role can't do more than deploy.

25 **HIGH** Known-vulnerable dependencies and images are detected and triaged

- Automated dependency alerts are on (Dependabot, Renovate, `npm audit`, `pip-audit`, `trivy fs .`).
- Container images are scanned before deploy (`trivy image <image>`, or ECR / Defender registry scanning).
- **PASS:** there's a working alert path and **no critical, internet-reachable vulnerabilities older than 14 days**.

Prioritize your findings

This is a **prioritization aid**, not a **security score**. A short list of FAILs doesn't mean you're safe, and UNKNOWNs are gaps in what you saw, not passes.

1. **Any CRIT FAIL:** contain now (see the severity table). Treat a **CRIT UNKNOWN** as open until you get the evidence.
2. **Anything internet-facing:** **HIGH** findings on public endpoints (checks 16–22) come next.
3. **Check 11:** make sure you'll know the next time something happens.

4. **Check 24:** make sure the pipeline isn't the way in.
5. Everything else, by severity and owner.

Record coverage with your results: how many checks you rated PASS, FAIL, N/A and UNKNOWN, which regions and subscriptions you covered, and which permissions were missing. The next audit should have fewer UNKNOWNs than this one.

Where next

The **SpaceGhostKilla Cloud Security Field Manual** turns each of these checks into a remediation workflow: fixes, guardrails that prevent regressions, and AWS, Azure and Cloudflare examples.

→ spaceghostkilla.com/#store

Only assess systems you own or are explicitly authorized to test. This checklist is provided for defensive purposes, as is, without warranty.